Long Short Term Memory Networks

CSE 5526 - Intro to Neural Networks
Lecture 17

Instructor: Donald S. Williamson


@TheASPIREGrpOSU


theaspiregroup_osu


The ASPIRE Group @ OSU

Agenda and Learning Outcomes

Upcoming Topics

- **Topics**:
  - Alternative RNN architectures
  - Limitations of RNNs and training issues
  - Discuss Long Short-Term Memory (LSTM) cells
- **Announcement**: Quiz #6, due by Tuesday, March 25th @ 11:59pm

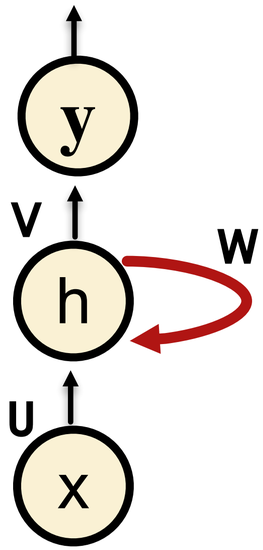
- **Learning Objectives: You the student will be able to:**
  - Understand and describe Long Short-Term Memory (LSTM) RNNs
  - Understand how LSTMS overcome issues with RNNS

Recap: Recurrent Hidden Units (RHU)

Forward pass of RNN with RHU: Unfold over time

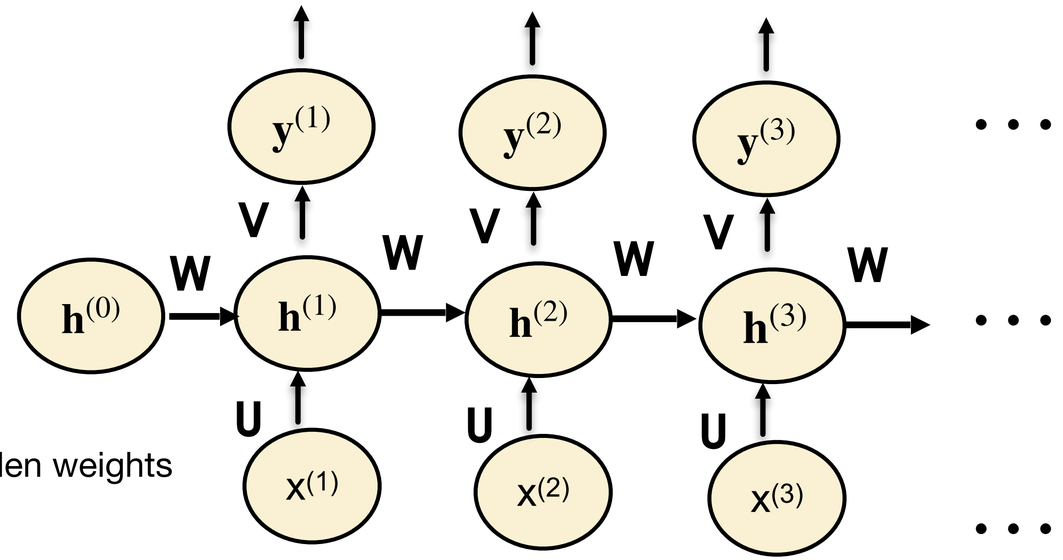
- **Example of *sequence-to-sequence* mapping**

- Assume a sequence of length T (e.g. there are T separate input features and T separate corresponding labels - $x^{(t)}$ represents a frame of a video)



Unfolding in time

=>



- **U**: input weights
- **W**: hidden-to-hidden weights
- **V**: output weights
- **y:** network output
- **h:** hidden layer output
- **x**: network input

$$\mathbf{v}^{(t)} = \mathbf{U}\mathbf{x}^{(t)} + \mathbf{b} + \mathbf{W}\mathbf{h}^{(t-1)}$$

$$\mathbf{h}^{(t)} = \phi(\mathbf{v}^{(t)})$$

$$\mathbf{y}^{(t)} = \phi(\mathbf{V}\mathbf{h}^{(t)} + \mathbf{c})$$

b and ***c*** are bias terms
$\mathbf{h}^{(0)}$ is initialized to 0

Image adopted from *Goodfellow, 2016*

Recap: Weight Update with RHU

Need to compute gradient with respect to each weight

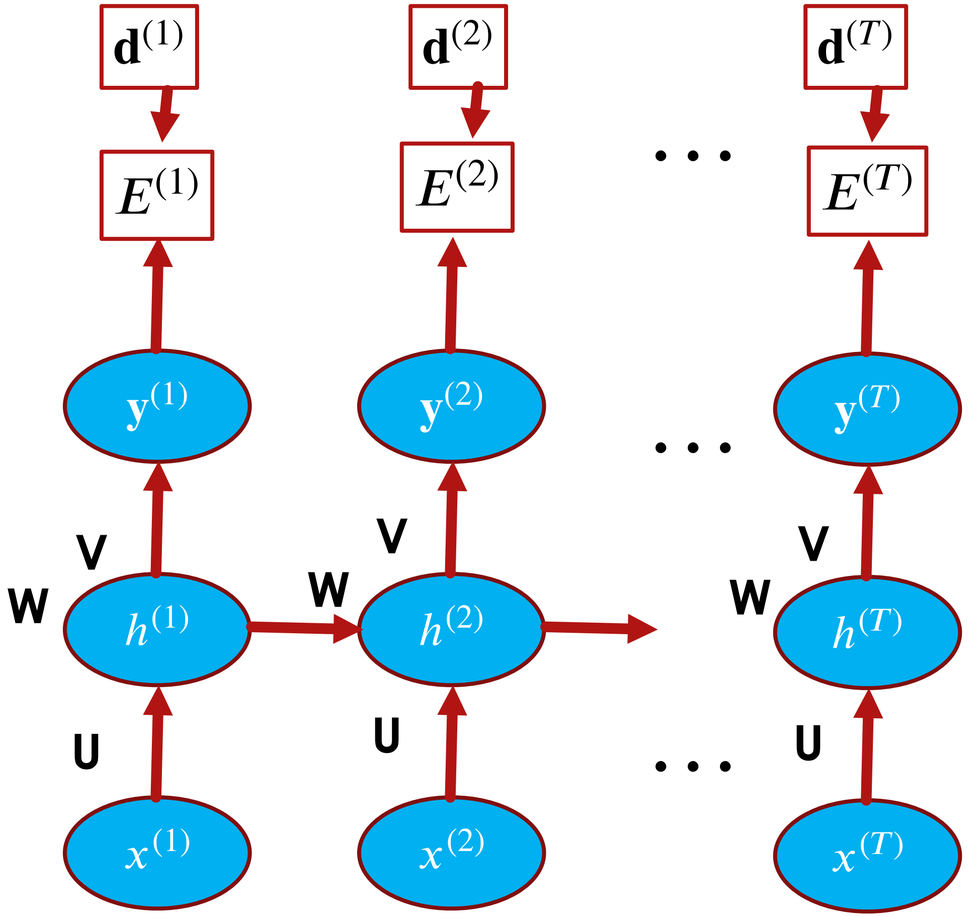
- For a single-hidden layer RNN
  - **Output layer:**

$$V(n+1) = V(n) + \eta_V \sum_{t=1}^{T} \delta_{\mathbf{y}}^{(t)} \mathbf{h}^{(t)}$$

  - **Hidden Layer:**

$$W(n+1) = W(n) + \eta_W \sum_{t=1}^{T} \delta_h^{(t)} \mathbf{h}^{(t-1)}$$

  - **Input Layer:** $U(n+1) = U(n) + \eta_U \sum_{t=1}^{T} \delta_h^{(t)} \mathbf{x}^{(t)}$

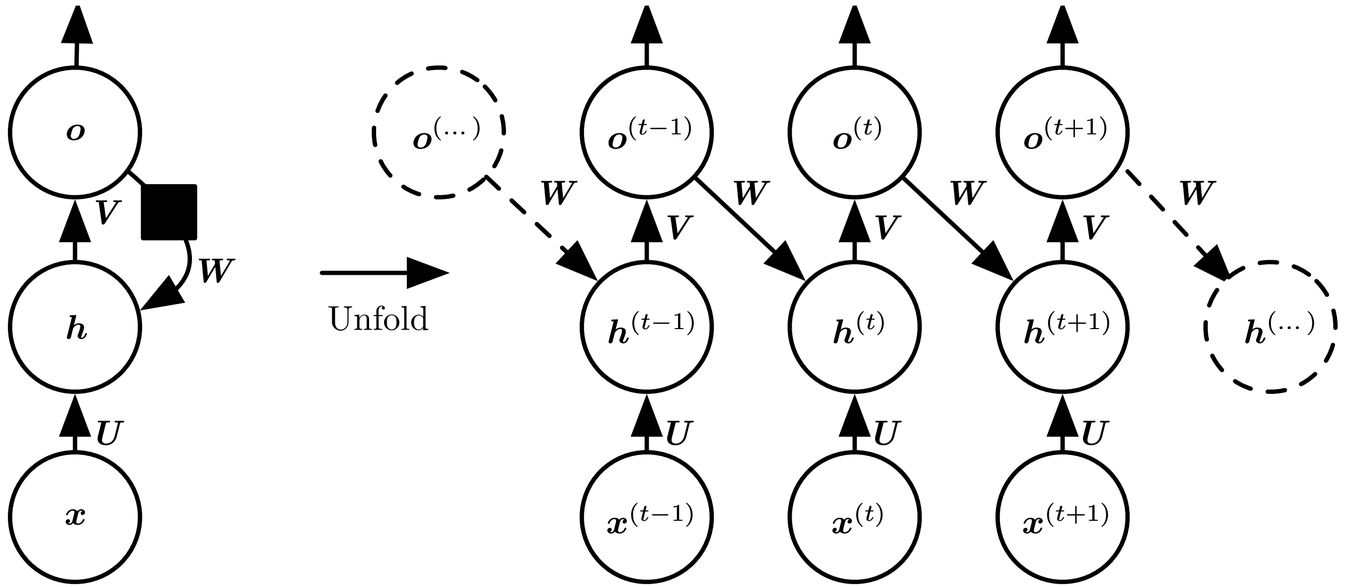


Alternative RNN architectures

RNN with Recurrent Output Units

- Recurrent connection from prior output to current hidden unit

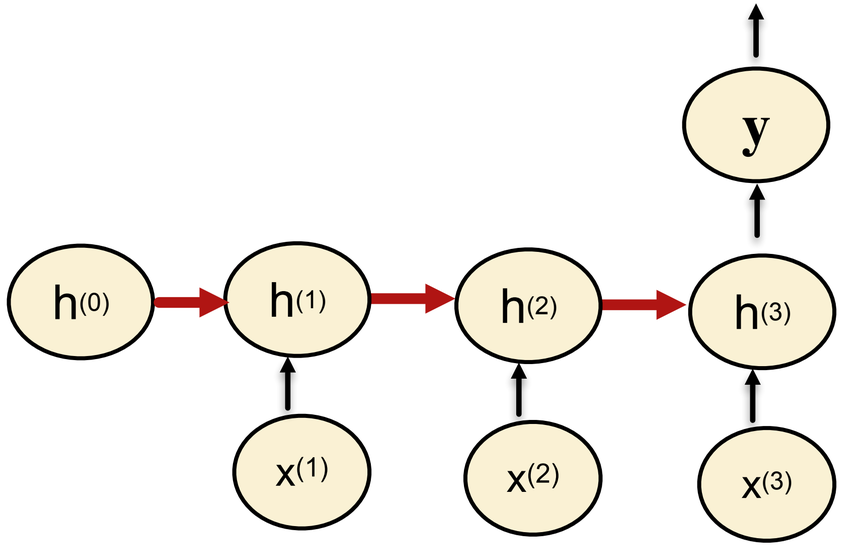
- This RNN is less “powerful” than the prior.
  - This network only retains output information
  - The prior network can choose to put any information into the hidden state and pass it to the future.



Alternative RNN architectures

Sequential Input, Single Output

- Time unfolded RNN with a single output at the end of the sequence.

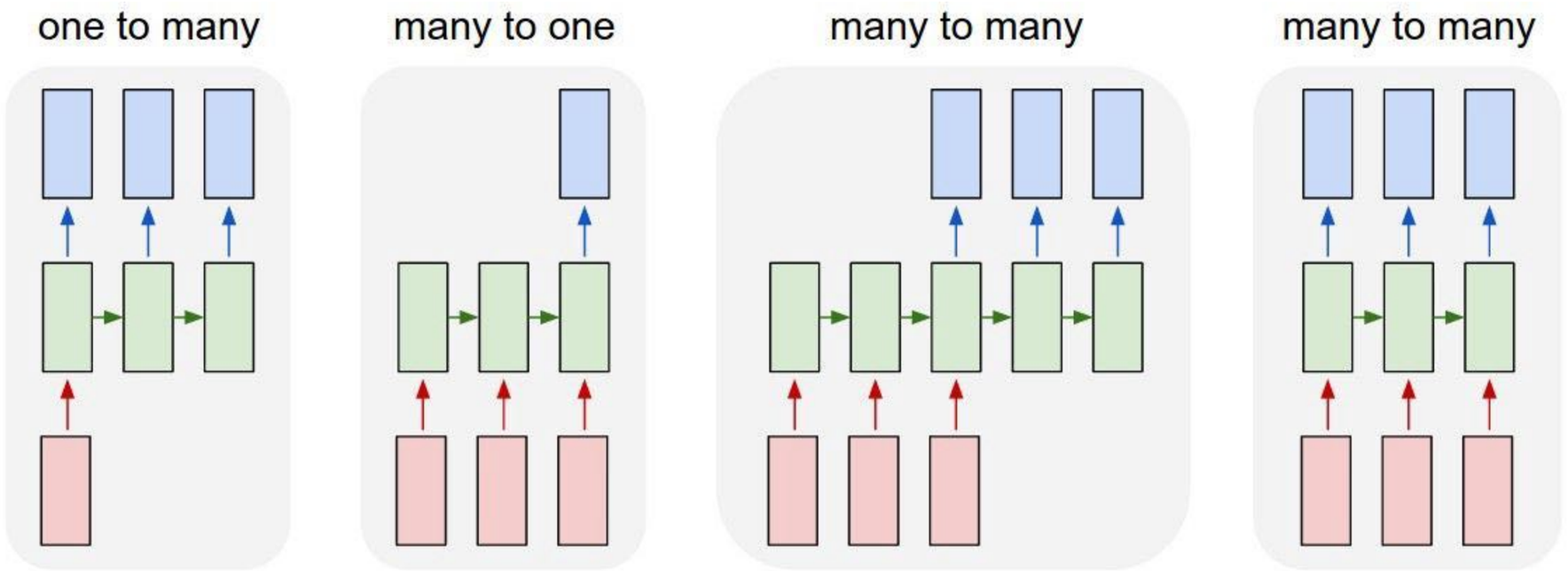


- This network is useful for summarizing a sequence and producing a fixed-size representation, which may be useful for further processing

- What application could this be useful for?

Other RNN Configurations

Think-Pair-Share



Summarizing RNN Training

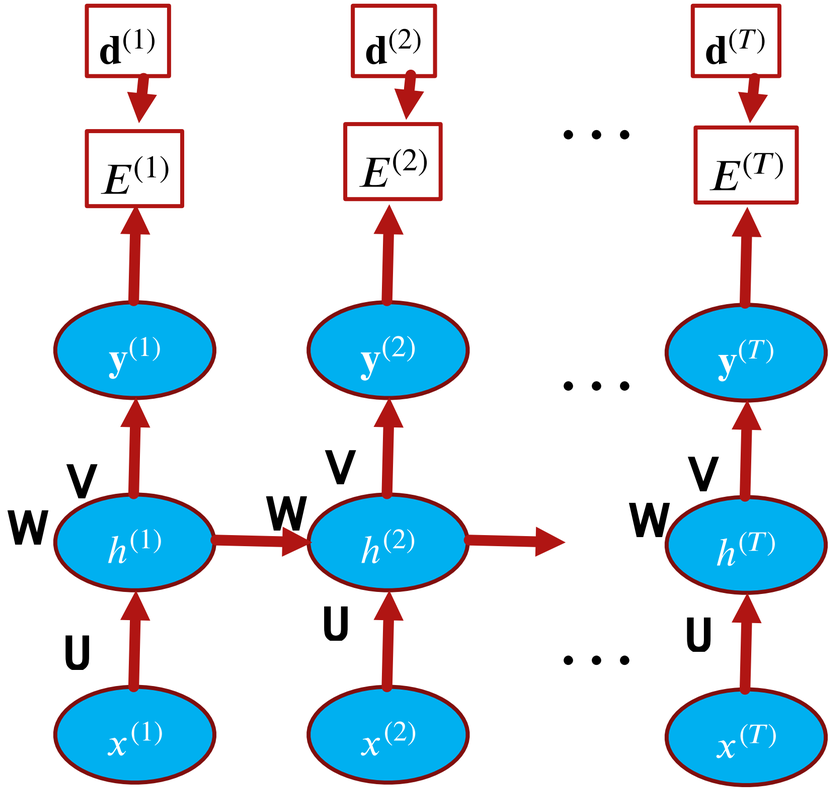
- Essentially, an RNN neuron is copied over time (unrolling or unfolding), with different inputs at different time steps
- Key difference from traditional backpropagation is the use of shared weights
  - Pretend that the weights are not shared and apply normal backpropagation to compute the gradients with respect to each copy of the weights
  - Summate (or average) the gradients over the various copies of each weight copy
  - Perform gradient descent update
- This algorithm is referred to as ***backpropagation through time (BPTT)***

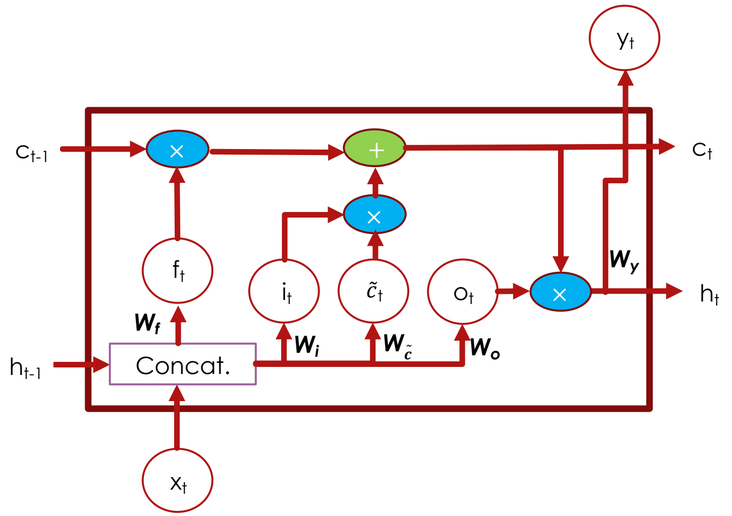
Questions

Limitations of RNNs

- **Training RNNs is difficult:** vanishing/exploding gradients

- **RNNs do not capture long-term dependencies**
  - They have short memories. Prior example captures a single time delay
  - Several data applications, including speech, have long-term dependencies
    - e.g., interactions of words that are several steps apart
    - ***Long-short term memory (LSTM)*** RNNs are used instead





Long Short-Term Memory (LSTM) networks

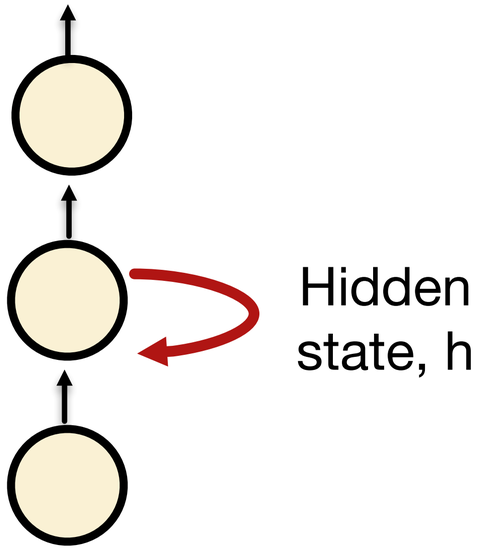
- **LSTMs are a special kind of RNN**
  - Capable of learning long-term dependencies (practically, not just theoretically)
  - They incorporate “gates” to help with learning short and long-term dependencies

- **Introduced by Hochreiter and Schmidhuber**
  - S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, vol. 9, pp. 1735-1780, 1997
  - **Refined by Gets, Schmidhuber, and Cummins**
    - F.A. Gers, J. Schmidhuber, and F. Cummins, “Learning to Forget: Continual Prediction with LSTM,” Neural Computation, vol. 12, pp. 2451-2471, 2000
    - Current implementations generally use this version

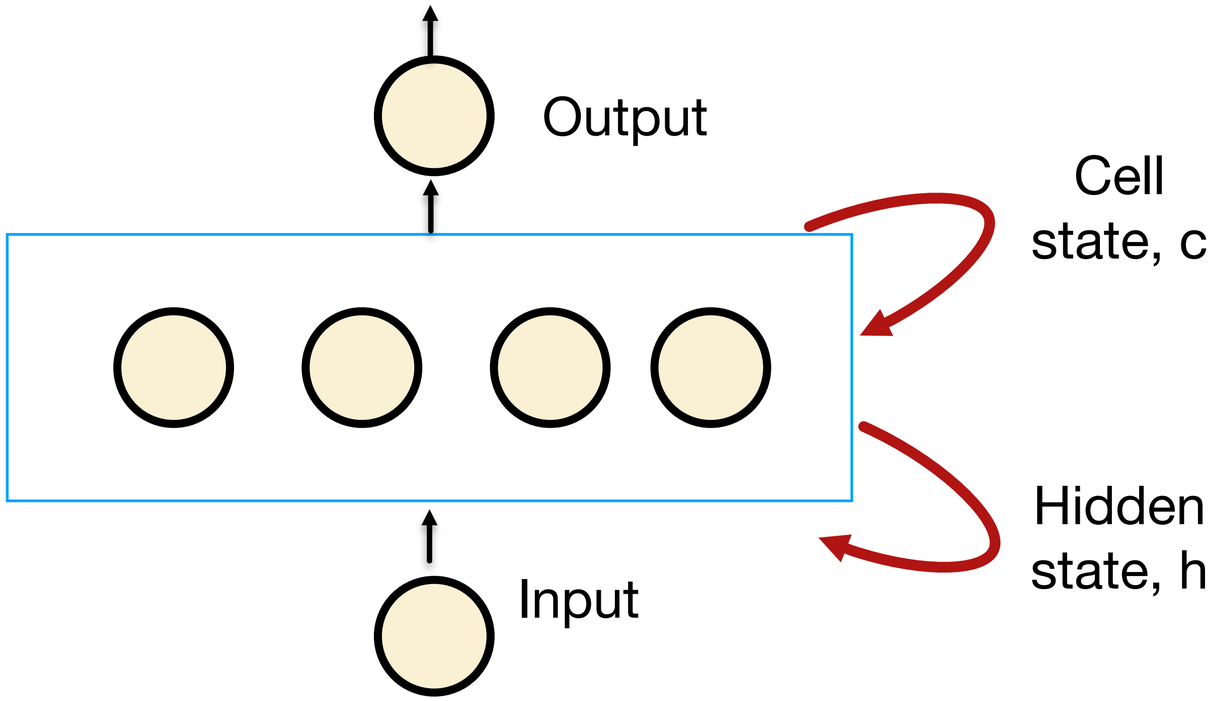
RNN Neuron vs LSTM Cell

A RNN neuron has a “single” recurring unit within the layer

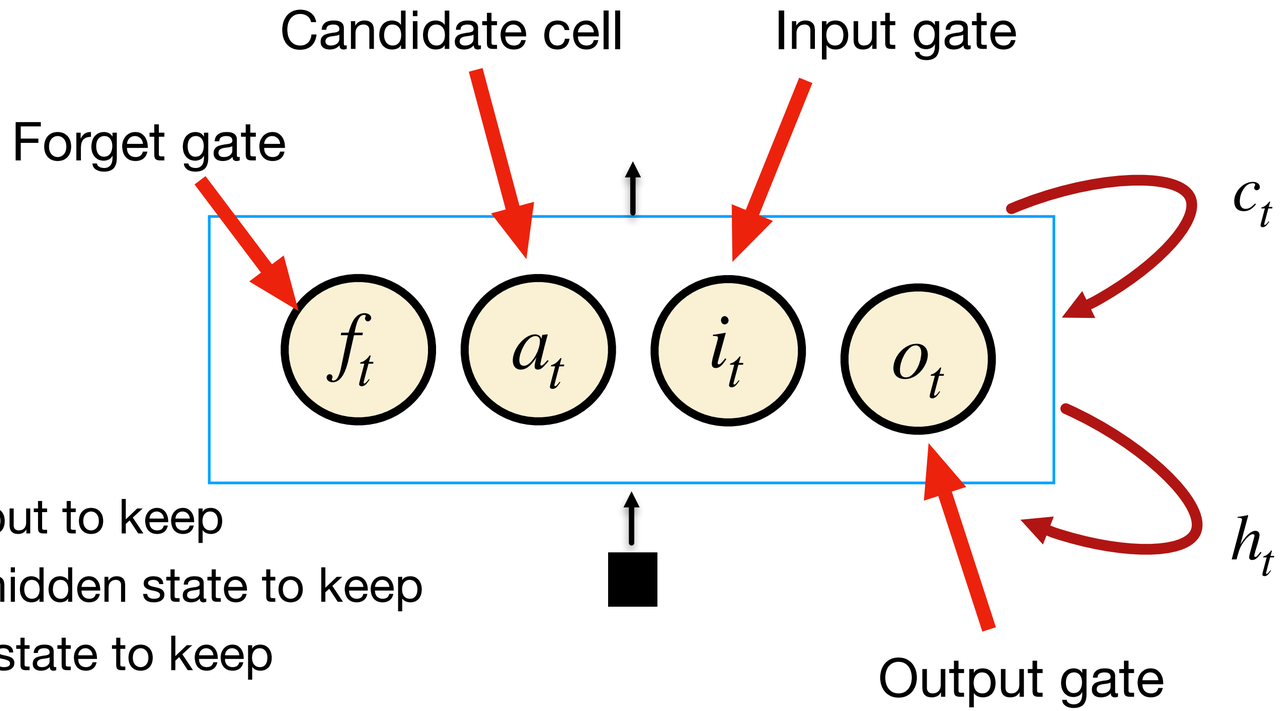
Ex. Hidden layer Recurrency



A LSTM cell has four interacting units within each neuron. This unit is called a ***memory cell***.



A LSTM Cell



- A LSTM Cell contains three gates:
  - Forget, input, and output gates
  - It also has a candidate cell output

- Gates decide which information to let through
  - **Input gate** determines how much of the input to keep
  - **Output gate** determines how much of the hidden state to keep
  - **Forget gate** controls how much of the cell state to keep

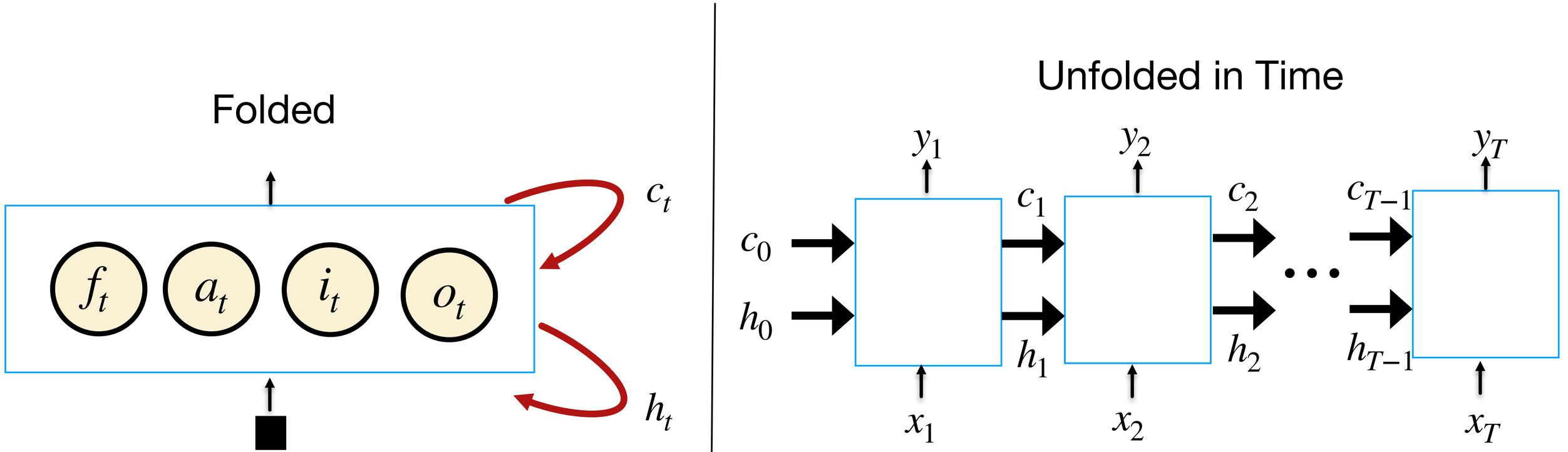
- The **candidate cell** (along with input gate) controls how much to add to the cell

- These all vary with time.

- This architecture lets the unit learn longer-term dependencies (e.g., cell state, c_t).

An Unfolded LSTM Cell

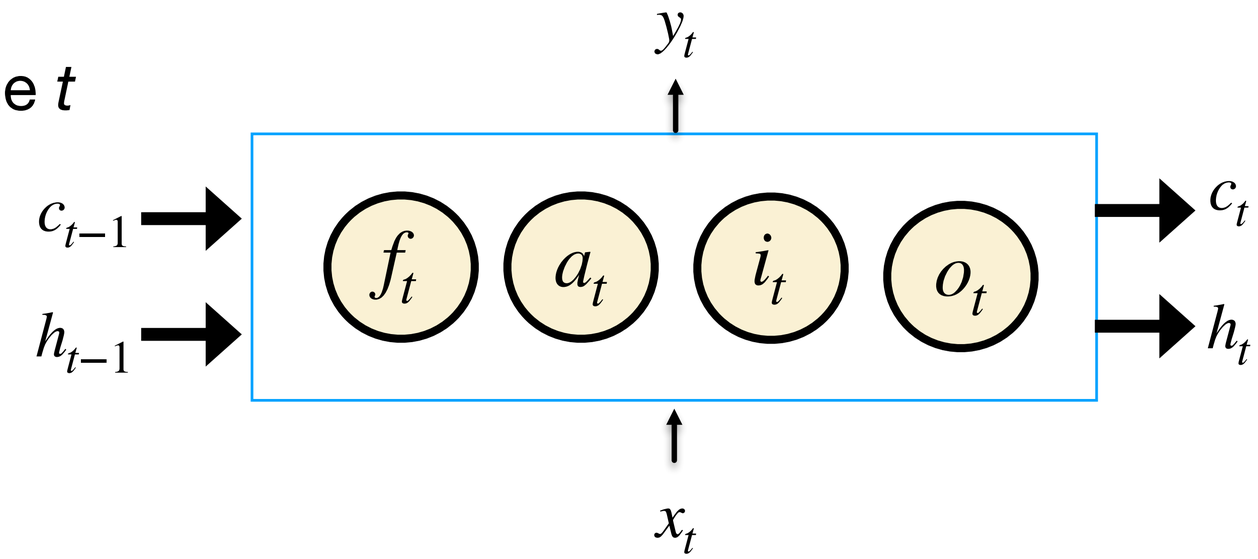
Example of sequence-to-sequence mapping



A LSTM Cell Through Time

Unrolled, at time *t, for sequence-to-sequence mapping*

- Let's look at the LSTM cell at time *t*
- The LSTM has three outputs at time *t*
  - Cell state
  - Hidden state
  - Output sequence element
- The LSTM cell has three inputs
  - Previous cell and hidden states
  - Input sequence at time t

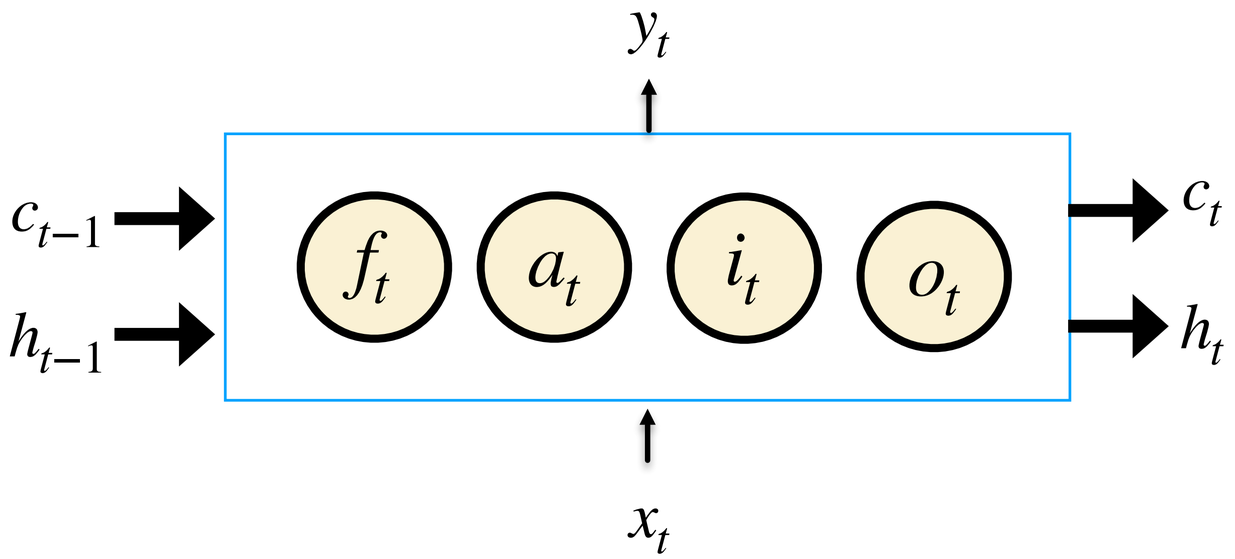


Gate Calculations

For sequence-to-sequence mapping

- The gates are calculated from the previous hidden state and current input

- The gates typically use sigmoid or hyperbolic tangent (e.g., tanh) activation functions



LSTM: Forward Pass

Input and gate computation

- Candidate Cell state:

$$\mathbf{a}^{(t)} = \tanh(\mathbf{U}_a\mathbf{x}^{(t)} + \mathbf{b}_a + \mathbf{W}_a\mathbf{h}^{(t-1)}) = \tanh(\mathbf{v}_a^{(t)})$$

- Input gate:

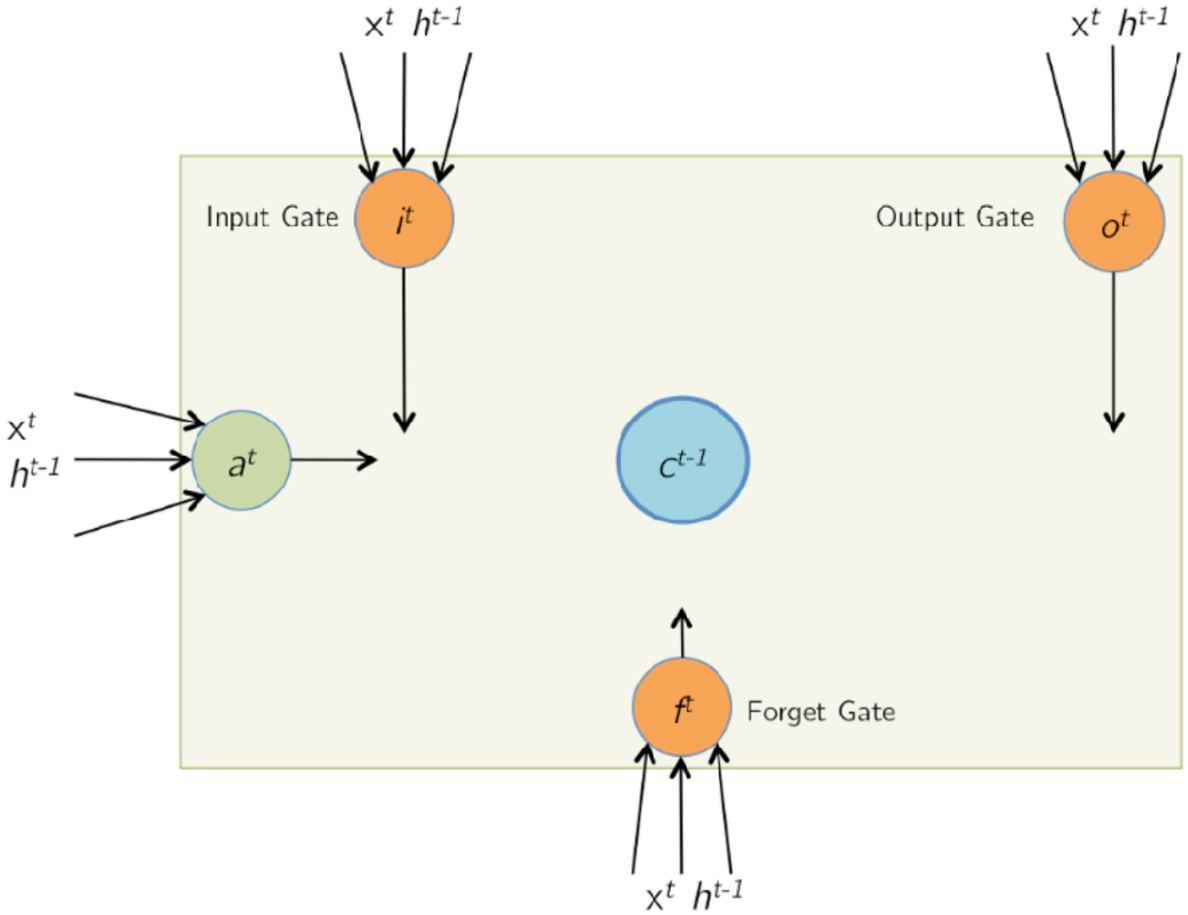
$$\mathbf{i}^{(t)} = \sigma(\mathbf{U}_i\mathbf{x}^{(t)} + \mathbf{b}_i + \mathbf{W}_i\mathbf{h}^{(t-1)}) = \sigma(\mathbf{v}_i^{(t)})$$

- Forget gate:

$$\mathbf{f}^{(t)} = \sigma(\mathbf{U}_f\mathbf{x}^{(t)} + \mathbf{b}_f + \mathbf{W}_f\mathbf{h}^{(t-1)}) = \sigma(\mathbf{v}_f^{(t)})$$

- Output gate:

$$\mathbf{o}^{(t)} = \sigma(\mathbf{U}_o\mathbf{x}^{(t)} + \mathbf{b}_o + \mathbf{W}_o\mathbf{h}^{(t-1)}) = \sigma(\mathbf{v}_o^{(t)})$$



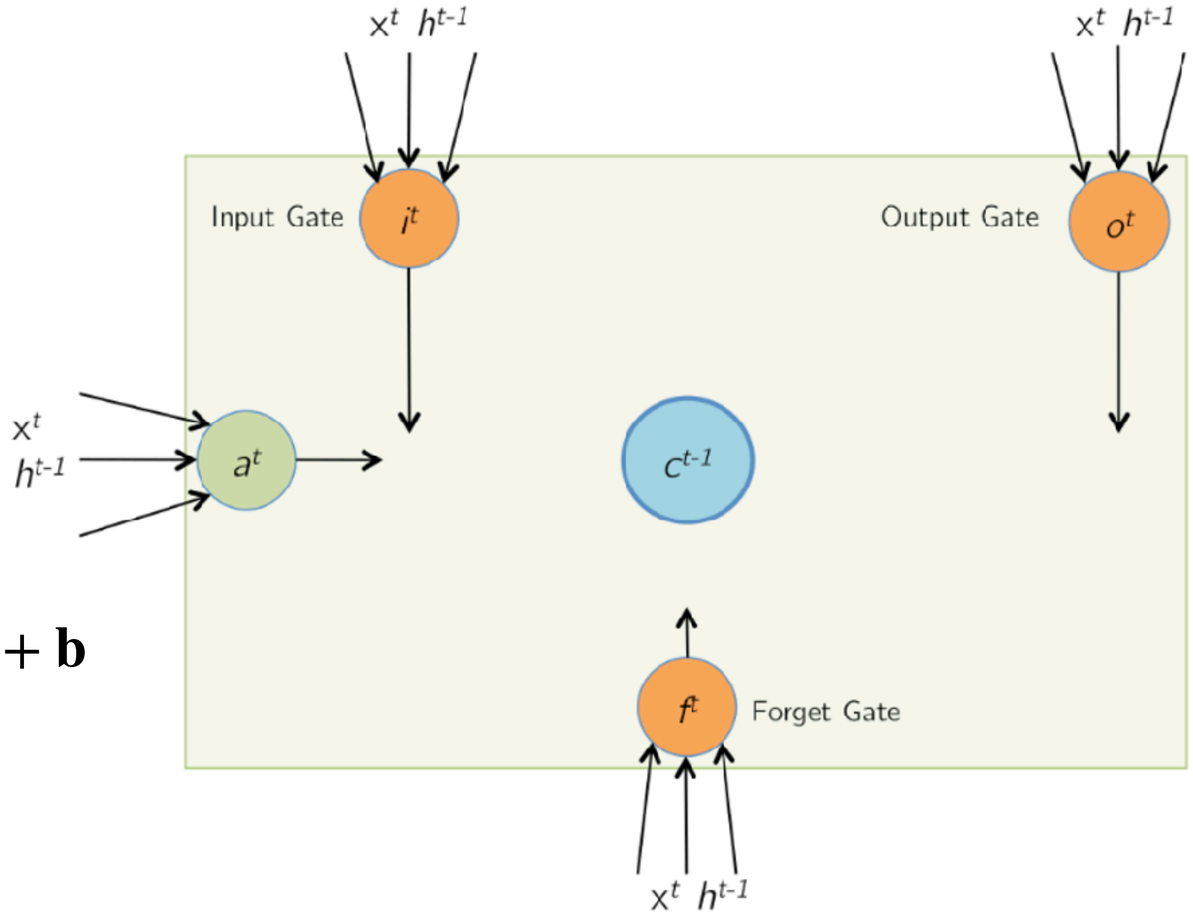
σ indicates a sigmoid activation

LSTM: Forward Pass

Input and gate computation

- In vector form for all the gates/cells:

$$\mathbf{v}^{(t)} = \begin{bmatrix} \mathbf{v}_a^{(t)} \\ \mathbf{v}_i^{(t)} \\ \mathbf{v}_f^{(t)} \\ \mathbf{v}_o^{(t)} \end{bmatrix} = \begin{bmatrix} \mathbf{W}_a & \mathbf{U}_a \\ \mathbf{W}_i & \mathbf{U}_i \\ \mathbf{W}_f & \mathbf{U}_f \\ \mathbf{W}_o & \mathbf{U}_o \end{bmatrix} \begin{bmatrix} \mathbf{h}^{(t-1)} \\ \mathbf{x}^{(t)} \end{bmatrix} + \begin{bmatrix} \mathbf{b}_a \\ \mathbf{b}_i \\ \mathbf{b}_f \\ \mathbf{b}_o \end{bmatrix} = \mathbf{W}\mathbf{I}^{(t)} + \mathbf{b}$$



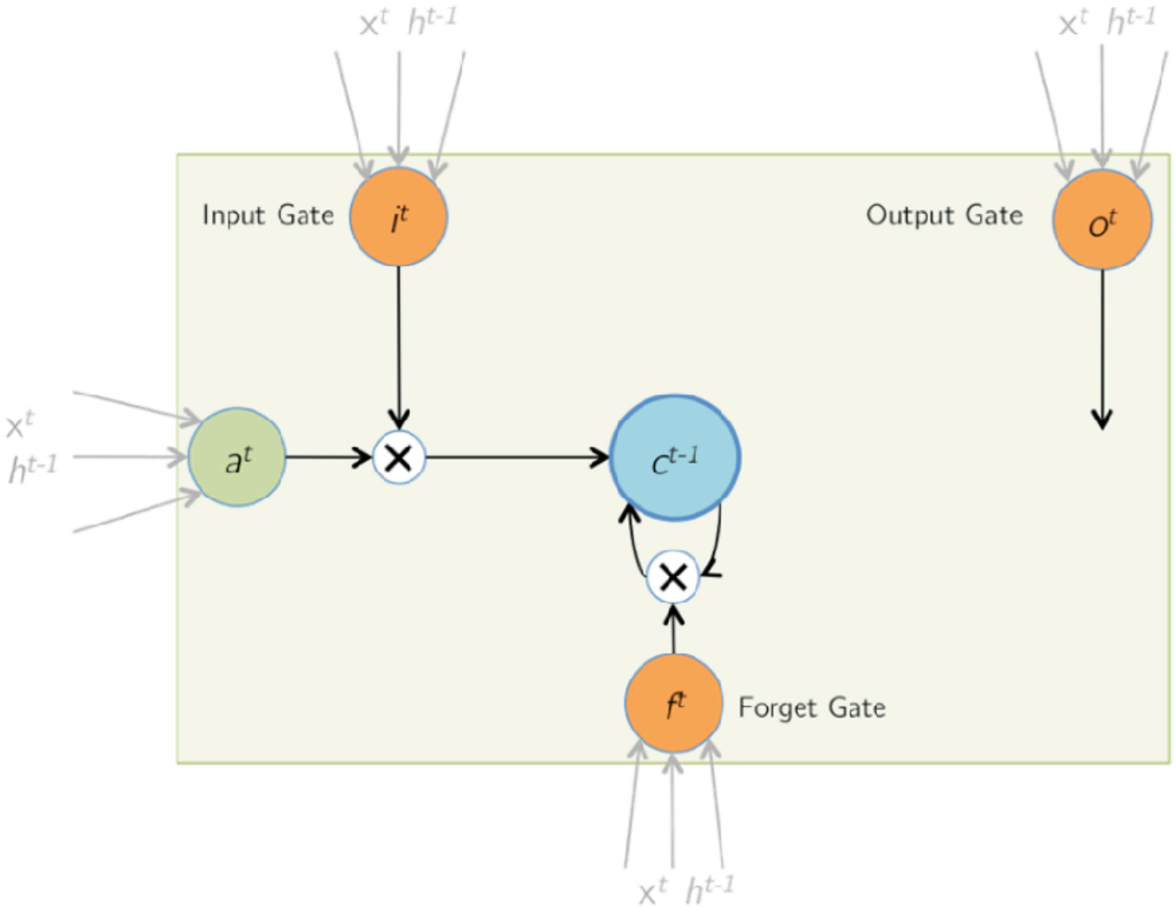
LSTM: Forward Pass

New Cell State

- **Memory update**: the state of the unit is updated to the latest values

$$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \odot \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \odot \mathbf{a}^{(t)}$$

- Symbol $\odot$ denotes elementwise product
- The forget gate determines how much of previous cell state to “forget”
- The input gate decides how much of the candidate values will be added



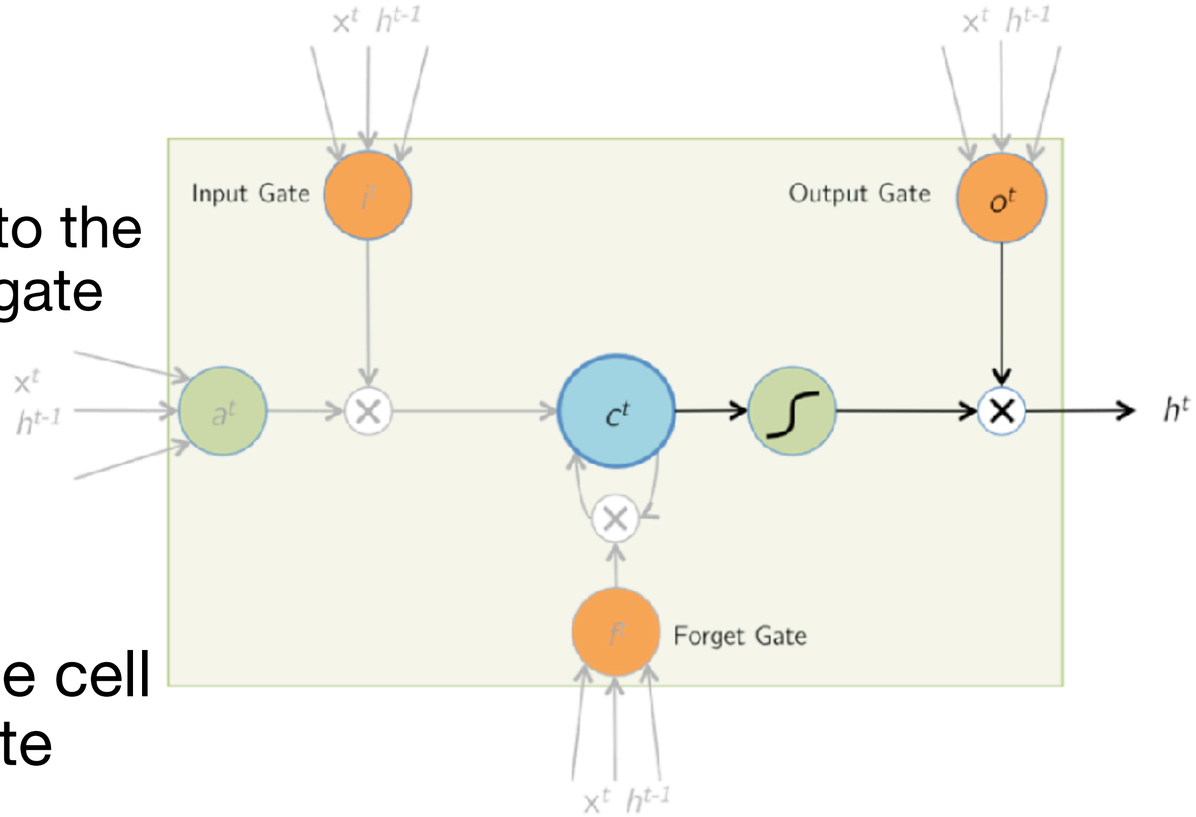
LSTM: Forward Pass

Hidden State Calculation

- The LSTM unit updates the hidden state by applying an activation function (nonlinearity) to the cell state, and weighing that with the output gate

$$\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \odot \tanh(\mathbf{c}^{(t)})$$

- Symbol ⊙ denotes elementwise product
- The output gate decides how much of the cell state will be outputted for the hidden state
- The hidden state has values between -1 and 1



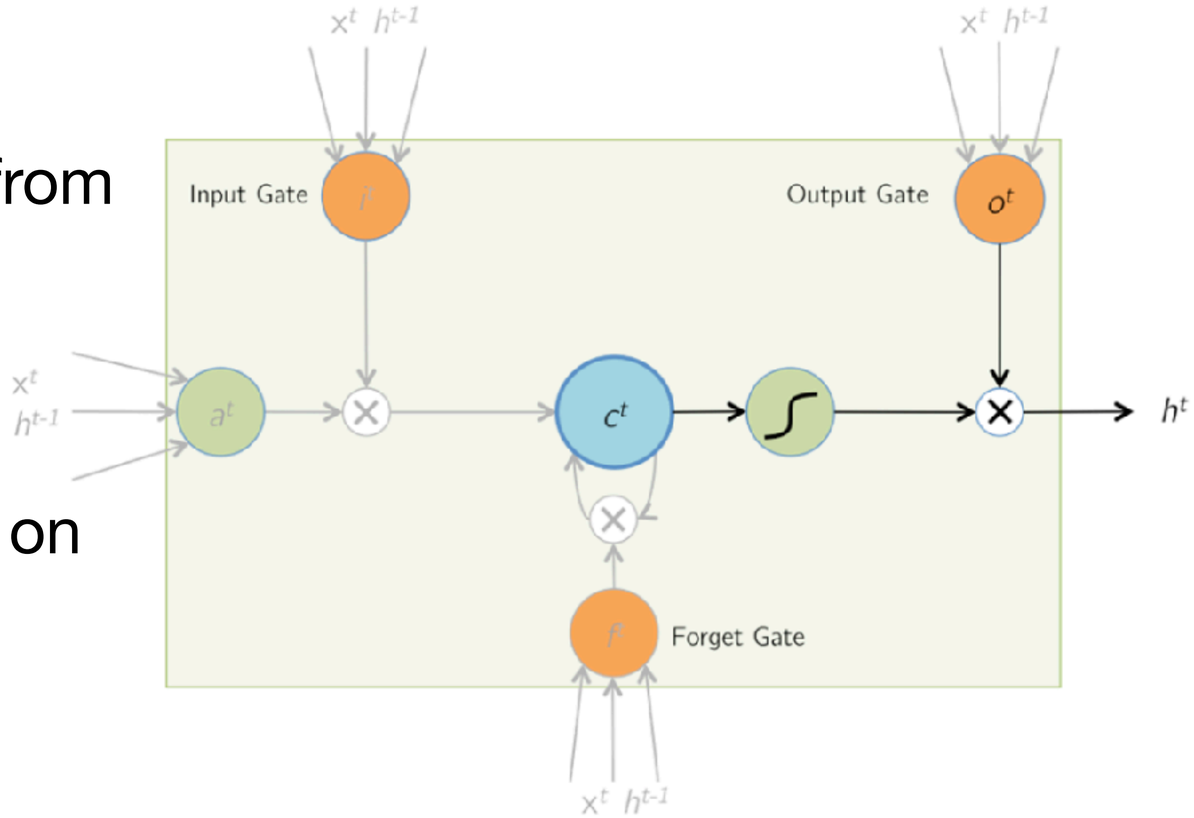
LSTM: Forward Pass

[Optional] Output Calculation

- The output of the cell, $y^{(t)}$, is calculated from the hidden state

$$y^{(t)} = \phi(\mathbf{W}_y h^{(t)} + b_y)$$

- The activation function used will depend on the goal of the network



Questions

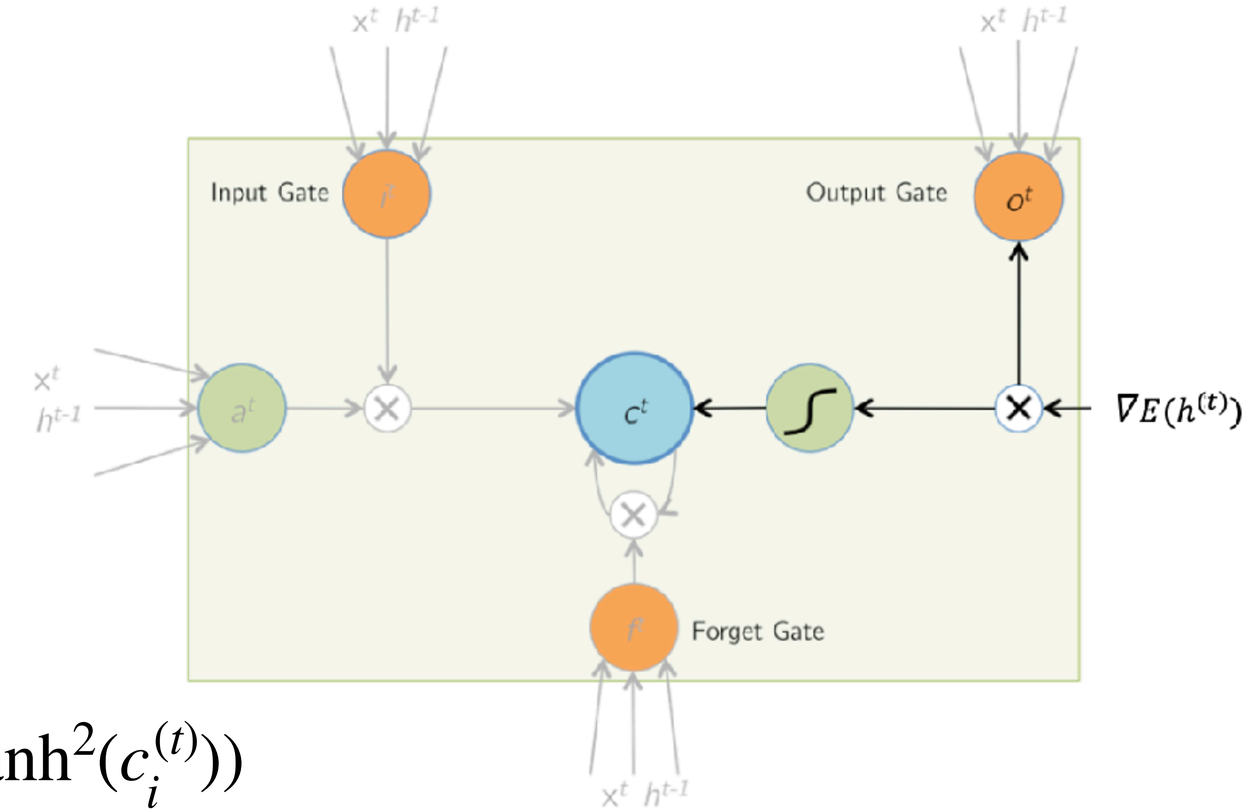
LSTM: Backward Pass

$$\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \odot \tanh(\mathbf{c}^{(t)})$$

- The cell state at time t, $\mathbf{c}^{(t)}$, receives gradients from $\mathbf{h}^{(t)}$as well as the next cell state $\mathbf{c}^{(t+1)}$.
- At time step t, these two gradients are accumulated before being backpropagated to the layers below the cell and the previous time steps
- Given $\nabla E(\mathbf{h}^{(t)}) = \dfrac{\partial E}{\partial \mathbf{h}^{(t)}}$, find $\nabla E(\mathbf{c}^{(t)})$

$$\frac{\partial E}{\partial c_i^{(t)}} = \frac{\partial E}{\partial h_i^{(t)}} \frac{\partial h_i^{(t)}}{\partial c_i^{(t)}} = \nabla E(h_i^{(t)}) o_i^{(t)} (1 - \tanh^2(c_i^{(t)}))$$

Thus, $\nabla E(\mathbf{c}^{(t)}) \mathrel{+}= \nabla E(\mathbf{h}^{(t)}) \odot \mathbf{o}^{(t)} \odot (1 - \tanh^2(\mathbf{c}^{(t)}))$



- Symbol $+=$

  indicates this gradient is added to the gradient from $(t+1)$ (i.e. gradient accumulation)

LSTM Training

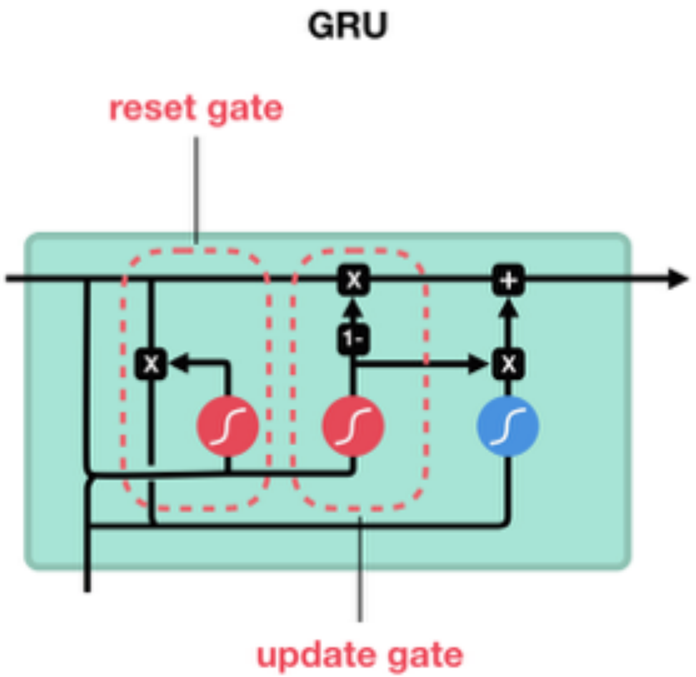
- Backpropagation through time (BPTT) is used to train LSTMs
- Previously defined BPTT algorithm must be modified for the additional weight and bias terms
- Derivation and computation of gradients follows the same framework (e.g. partial derivatives)
- We will NOT go through the specifics in class. See Gers et al. (2000) for details

LSTM Variants and Design

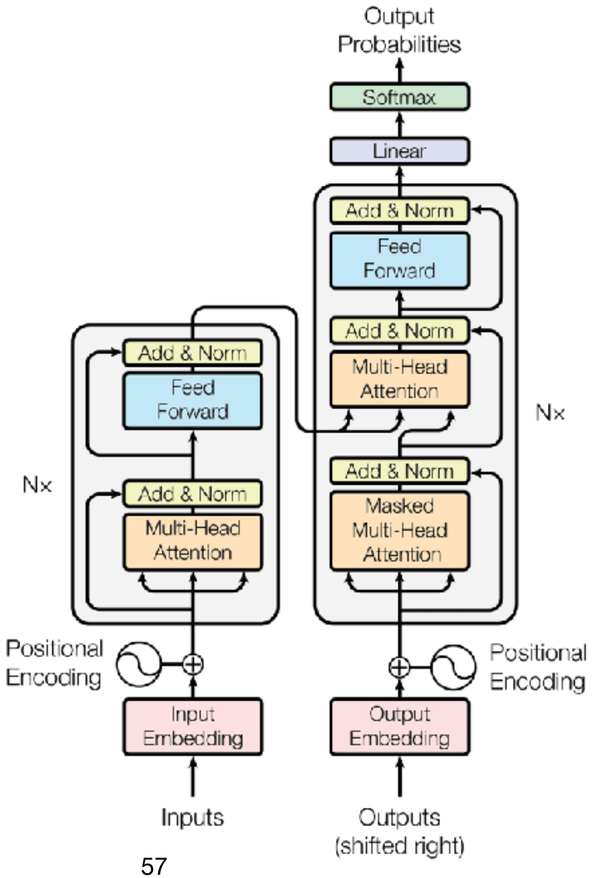
- We presented a simplified (single) view of LSTMs (single cell, single hidden layer)
- LSTMs obviously can be more complicated
  - Multiple hidden layers, and multiple LSTM cells per hidden layer
  - Multi-dimensional inputs and outputs
  - Used for sequence-to-sequence, or sequence-to-single predictions
  - Reverse direction, bi-directional
- Design considerations
  - Number of layers and units
  - Length of the sequence
  - Activation function
  - Other neural network decisions (e.g. momentum, learning rate, dropout, optimization, etc.)

Other Sequence Approaches

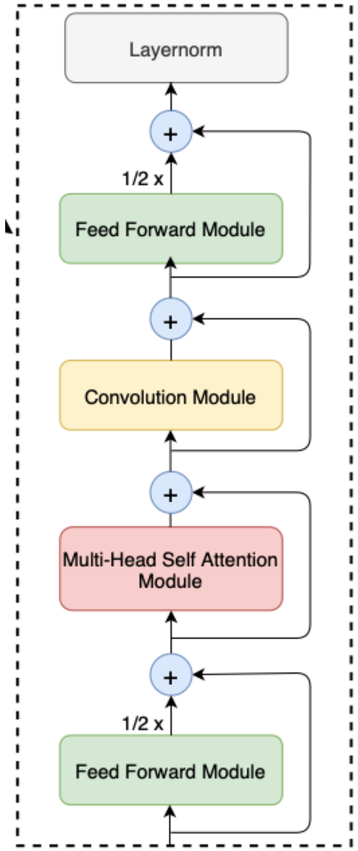
Gated Recurrent Unit



Transformer



Conformer



Summary

- RNNs are uniquely suited for sequence modeling and temporal (dynamical) tasks
- RNN training is more difficult than training a feedforward net
- Gated RNNs, particularly RNNs with LSTM cells, are very successful in real world applications
- However, attention (transformers) is currently the sequencing network of choice (more on this later)